MONASH University
Information Technology

FIT3159
Computer Architecture

Combinatorial Logic Optimisation / Logic Circuits

Dr Chetan Arora, FHEA
Chetan.Arora@monash.edu

Dr Charith Jayasekara, FHEA, SMIEEE, MACS CP, MIEAust
Charith.Jayasekara@monash.edu

Dr Carlo Kopp, SMIEEE, AFAIAA, FLSS, Peng
Carlo.Kopp@monash.edu

Faculty Information Technology,
Monash University
Australia

Why Simplification and Optimisation of Logic?

- **Hardware "cost" is typically proportional to circuit complexity – twice as many logic expressions means twice as many gates:**
  - Twice as much real estate on a chip
  - Twice as much electrical power consumption
  - Twice as much heat dissipation
  - Twice as much cooling capacity
- **Desktop, deskside, server systems – cost, size and cooling are important.**
- **Handheld, notebook, portable systems – battery power drain matters critically.**
- **Cost is measurable directly in dollars per chip, and indirectly in the cost to support the chip with packaging, power, cooling etc.**

© Monash University, 2004-2025.

What is Boolean Optimisation Also Useful For?

- Basic purpose is to reduce costs of hardware, but it is useful in other areas.
- Thirty years ago logic design was done almost completely via a manual process, where expressions were translated into logic circuit diagrams, used in turn to design actual hardware.
- Current approach is to use design software, where logic is defined by expressions, or schematic diagrams, and then compiled into minimised programming data for programmable logic devices, or masks for chip layout.
- Understanding minimisation aids in isolating problems when using design software.
- In writing complex code, complicated decision logic can be better understood and simplified.

© Monash University, 2004-2025.

MONASH University
Information Technology

Optimisation of Combinatorial Logic

Logic and Computer Design Fundamentals, 4e, PPT Slides
Charles Kime & Thomas Kaminski
© 2008, Pearson Education, Inc

Boolean Function Optimization

- Minimizing the gate input (or literal) cost of a (a set of) Boolean equation(s) reduces circuit cost.
- We choose gate input cost.
- Boolean Algebra and graphical techniques are tools to minimize cost criteria values.
- Some important questions:
  - When do we stop trying to reduce the cost?
  - Do we know when we have a minimum cost?
- Introduce a graphical technique using Karnaugh maps (K-maps, for short)

© Monash University, 2004-2025.

Karnaugh Maps (K-map)

- A K-map is a collection of squares
  - Each square represents a minterm
  - The collection of squares is a graphical representation of a Boolean function
  - Adjacent squares differ in the value of one variable
  - Alternative algebraic expressions for the same function are derived by recognizing patterns of squares
- The K-map can be viewed as
  - A reorganized version of the truth table
  - A topologically-warped Venn diagram as used to visualize sets in algebra of sets

© Monash University, 2004-2025.

Some Uses of K-Maps

- **Provide a means for:**
  - **Finding optimum or near optimum**
    - **two-level AND/OR and OR/AND circuit implementations**

    for functions with small numbers of variables
  - **Visualizing concepts related to manipulating Boolean expressions, and**
  - **Demonstrating concepts used by computer-aided design programs to simplify large circuits**

© Monash University, 2004-2025.

Two Variable Maps

- **A 2-variable Karnaugh Map:**
  - Note that minterm m0 and minterm m1 are "adjacent" and differ in the value of the variable y
  - Similarly, minterm m0 and minterm m2 differ in the x variable.
  - Also, m1 and m3 differ in the x variable as well.
  - Finally, m2 and m3 differ in the value of the variable y

	$y = 0$	$y = 1$
$x = 0$	$m_0 = \overline{x}\,\overline{y}$	$m_1 = \overline{x}\,y$
$x = 1$	$m_2 = x\,\overline{y}$	$m_3 = x\,y$

© Monash University, 2004-2025.

K-Map and Truth Tables

- The K-Map is just a different form of the truth table.
- Example – Two variable function:
  - We choose a,b,c and d from the set {0,1} to implement a particular function, F(x,y).

K-Map

	y = 0	y = 1
x = 0	a	b
x = 1	c	d

Function Table

Input Values (x,y)	Function Value F(x,y)
0 0	a
0 1	b
1 0	c
1 1	d

© Monash University, 2004-2025.

K-Map Function Representation

- **Example: F(x,y) = x**

F = x	y = 0	y = 1
x = 0	0	0
x = 1	1	1

- **For function F(x,y), the two adjacent cells containing 1's can be combined using the Minimization Theorem:**

$$F(x, y) = x\bar{y} + xy = x$$

© Monash University, 2004-2025.

K-Map Function Representation

- **Example: G(x,y) = x + y**

G = x+y	y = 0	y = 1
x = 0	0	1
x = 1	1	1

- **For G(x,y), two pairs of adjacent cells containing 1's can be combined using the Minimization Theorem:**

$$G(x,y) = \left(x\bar{y} + xy\right) + \left(xy + \bar{x}y\right) = x + y$$

Duplicate xy

© Monash University, 2004-2025.

MONASH University
Information Technology

Combinatorial Logic Circuits

Overview - Combinatorial Logic

- **Functions and functional blocks**
- **Rudimentary logic functions**
- **Decoding using Decoders**
  - Implementing Combinational Functions with Decoders
- **Encoding using Encoders**
- **Selecting using Multiplexers**
  - Implementing Combinational Functions with Multiplexers

© Monash University, 2004-2025.

Functions and Functional Blocks

- The functions considered are those found to be very useful in design
- Corresponding to each of the functions is a combinational circuit implementation called a *functional block*.
- In the past, functional blocks were packaged as small-scale-integrated (SSI), medium-scale integrated (MSI), and large-scale-integrated (LSI) circuits.
- Today, they are often simply implemented within a very-large-scale-integrated (VLSI) circuit or a Programmable Logic Device (PLD)

© Monash University, 2004-2025.

Rudimentary Logic Functions

- Functions of a single variable X
- Can be used on the inputs to functional blocks to implement other than the block's intended function

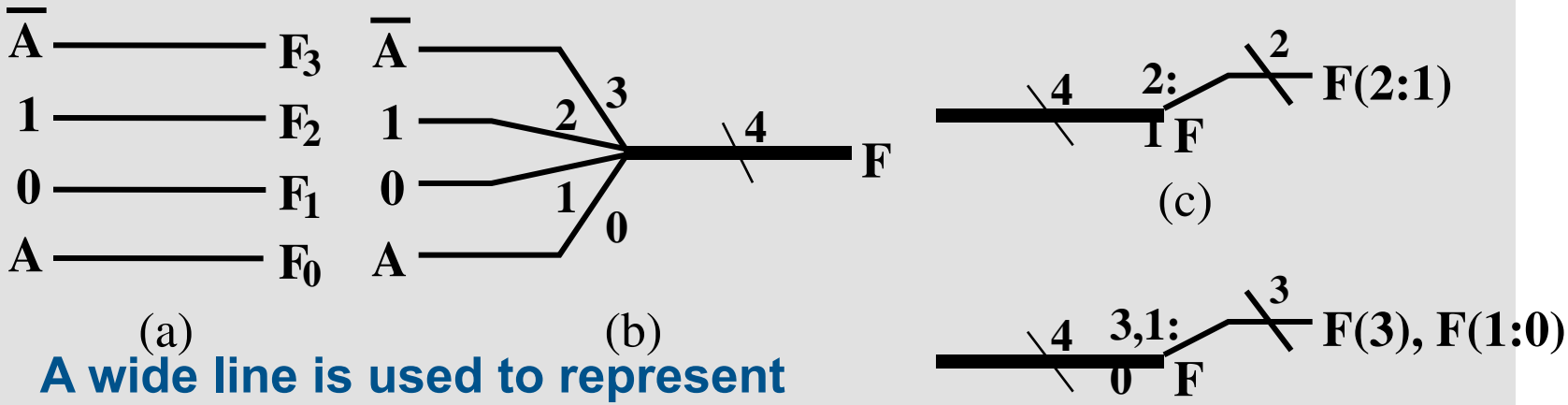
TABLE 4-1
Functions of One Variable

X	F = 0	F = X	F = $\bar{X}$	F = 1
0	0	0	1	1
1	0	1	0	1

© Monash University, 2004-2025.

Multiple-bit Rudimentary Functions

- **Multi-bit Examples:**



- **A wide line is used to represent a *bus* which is a vector signal**
- **In (b) of the example, F = (F_3, F_2, F_1, F_0) is a bus.**
- **The bus can be split into <u>individual bits</u> as shown in (b)**
- **<u>Sets of bits</u> can be split from the bus as shown in (c) for bits 2 and 1 of F.**
- **The sets of bits need not be continuous as shown in (d) for bits 3, 1, and 0 of F.**

© Monash University, 2004-2025.

Enabling Function

- *Enabling* permits an input signal to pass through to an output
- *Disabling* blocks an input signal from passing through to an output, replacing it with a fixed value
- The value on the output when it is disable can be Hi-Z (as for three-state buffers and transmission gates), 0 , or 1
- The Hi-Z (high impedance) state is when the output is electrically disconnected.
- In the other two states the output remains at 0 or 1 until the enabling signal is changed.

© Monash University, 2004-2025.

Decoding

- **Decoding - the conversion of an *n*-bit input code to an *m*-bit output code with $n \leq m \leq 2^n$ such that each valid code word produces a unique output code**
- **Circuits that perform decoding are called *decoders***
- **Here, functional blocks for decoding are:**
  1. called *n*-to-*m* line decoders, where $m \leq 2^n$, and
  2. generate 2^n (or fewer) minterms for the *n* input variables

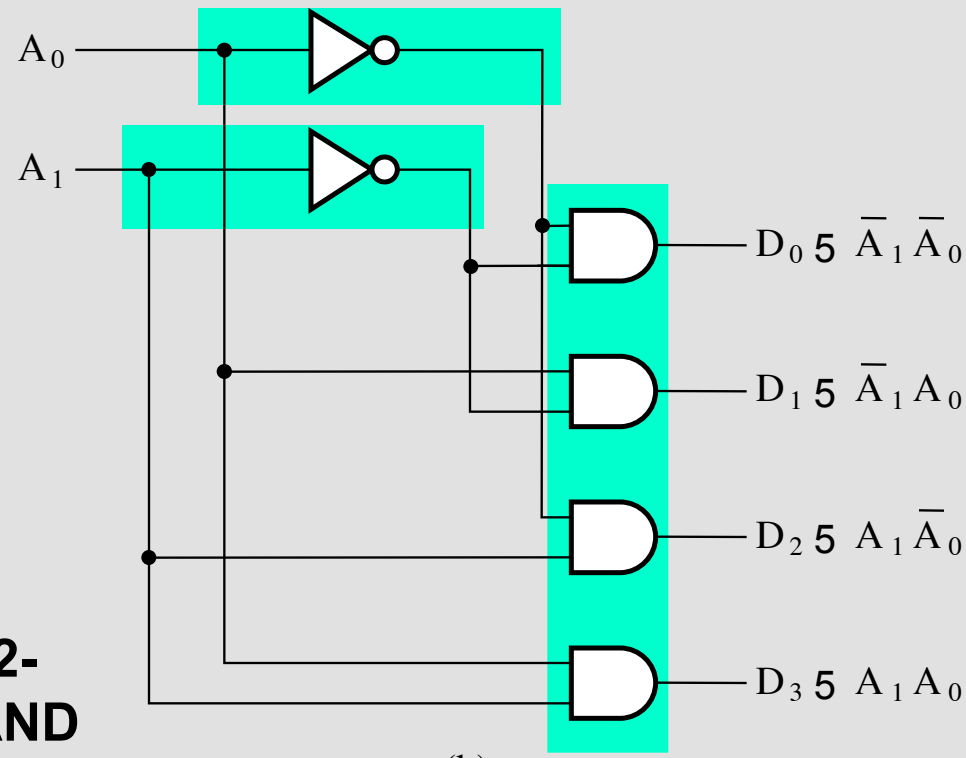
© Monash University, 2004-2025.

Decoder Examples

- **2-to-4-Line Decoder**

A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

(a)



(b)

- **Note that the 2-4-line made up of two 1-to-2-line decoders and 4 AND gates.**

© Monash University, 2004-2025.

Selecting

- **Selecting of data or information is a critical function in digital systems and computers**
- **Circuits that perform selecting have:**
  - A set of information inputs from which the selection is made
  - A single output
  - A set of control lines for making the selection
- **Logic circuits that perform selecting are called *multiplexers***
- **Selecting can also be done by three-state logic or transmission gates**

© Monash University, 2004-2025.

Multiplexers

- A multiplexer selects information from an input line and directs the information to an output line
- A typical multiplexer has *n* control inputs (S_{n-1}, … S_0) called *selection inputs*, 2^n information inputs (I_{2^n-1}, … I_0), and one output Y
- A multiplexer can be designed to have *m* information inputs with $m < 2^n$ as well as *n* selection inputs

© Monash University, 2004-2025.

2-to-1-Line Multiplexer

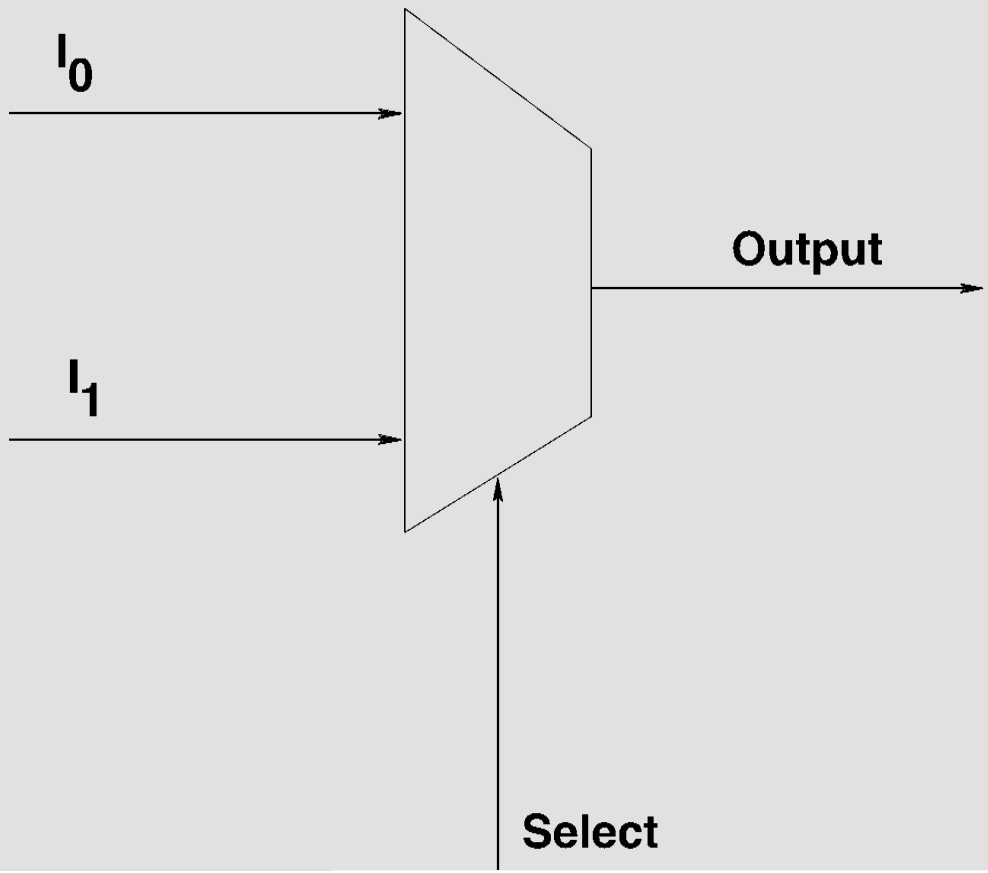
- **Since $2 = 2^1$, $n = 1$**
- **The single selection variable S has two values:**
  - S = 0 selects input I_0
  - S = 1 selects input I_1
- **The equation:**

$$Y = \bar{S} I_0 + S I_1$$

- **The circuit:**

© Monash University, 2004-2025.

2-to-1-Line Multiplexer



© Monash University, 2004-2025.

Hazards in Digital Logic

- Real logic circuits, whether made up from logic chips or embedded inside complex chips, cannot switch instantly – all are characterised by a real propagation delay.
- The result of real propagation delays is that some logic circuits will exhibit unwanted output behaviours, which are termed “hazards”.
- Hazards sometimes produce no impact on the behaviour of the circuit, but in some circuits can result in problems and incorrect logic behaviour.
- Robust design of logic requires that hazards be identified and the logic circuit changed to remove the hazard, or remove the conditions (inputs, timing) that cause the hazard to arise.

© Monash University, 2004-2025.

Types of Logic Hazard

- ***Static Hazards***: These arise when an input change produces a short transient reversal in the output, before the output settles to the state defined by its truth table.
- ***Dynamic Hazards***: These arise in complex logic circuits in which multiple static hazards may exist, and where the delays through multiple paths to the output are not the same. As a result more than one transient output change may occur before the output has settled.
- Static and Dynamic hazards can be usually fixed by altering the logic design, or the output can be buffered by a latch which is triggered after the output has settled.
- Other types of hazard exist, at a logic level, or a functional level and must be addressed if designs are to operate reliably.

© Monash University, 2004-2025.

Summary

- Minimising Logic
- Karnaugh Maps
- Enabling/Disabling
- Decoders
- Multiplexers
- Hazards

© Monash University, 2004-2025.